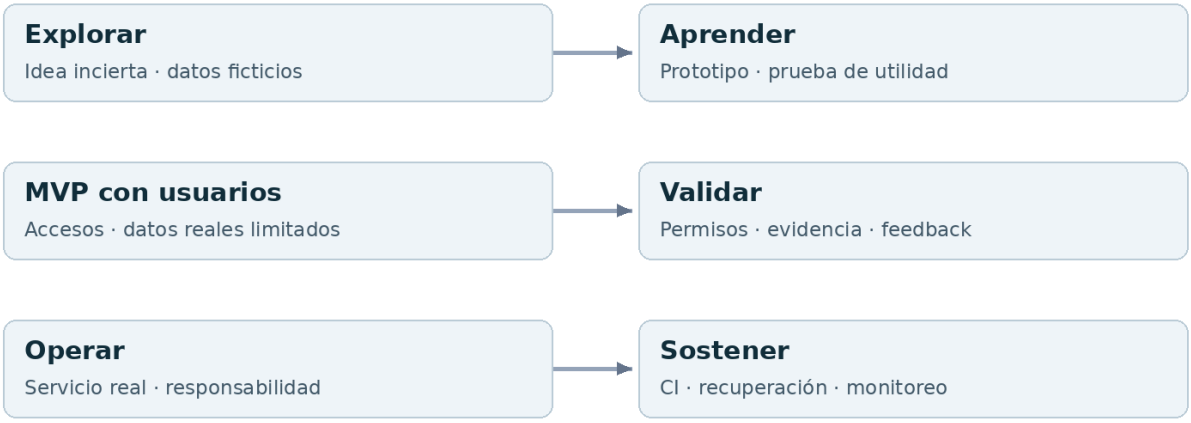
MANUAL PRÁCTICO / EDICIÓN REVISADA

Desarrollo con IA

Guía práctica de vibe coding

De la exploración de una idea a la operación de un producto. Tres recorridos para aprender, construir y publicar con decisiones explícitas y evidencia verificable.

Esta guía te ayuda a definir un problema, colaborar con un agente de programación y comprobar el resultado. Incluye plantillas, prompts, criterios de aceptación, seguridad, pruebas y fundamentos de operación.



La responsabilidad sigue siendo tuya

La IA propone y ejecuta dentro de los límites que le das. Tu trabajo es definir qué importa, revisar las decisiones y comprobar lo que afecta a las personas. Una interfaz convincente, un test en verde o una respuesta segura del agente no bastan por separado.

No necesitas toda la infraestructura desde el primer día. Sí necesitas distinguir un experimento de un servicio que otras personas usarán y saber qué evidencia exige cada uno.

Edición revisada en español · 30 de octubre de 2026. Las herramientas ilustran decisiones; no constituyen una clasificación del mercado. Consulta la documentación de las versiones que utilices.

01 / ORIENTACIÓN

Elige tu recorrido

Antes de elegir tecnologías, decide qué clase de trabajo vas a hacer. Los recorridos pueden sucederse, pero un prototipo no se convierte en producto operativo solo por publicarlo.

Recorrido	Qué intentas aprender	Condición de salida
Exploración	Si una idea merece más trabajo	Una observación útil y una decisión
MVP con usuarios	Si un flujo resuelve una necesidad real	Utilidad observada y riesgos controlados
Producto en operación	Si puedes sostener el servicio	Responsable, controles y recuperación comprobada

Exploración

Usa datos ficticios y el mínimo código necesario. Un prototipo local o una demo pública sin datos privados puede servir. Marca lo simulado: una pantalla de acceso sin autenticación real no debe aparentar que protege información.

MVP con usuarios

Acota usuarios, datos y funciones. Si hay cuentas o información privada, implementa los permisos antes de invitar personas. Define cómo recogerás feedback, atenderás errores y retirarás la prueba si deja de ser segura o útil.

Producto en operación

Exige controles reproducibles, despliegues rastreables, monitoreo, recuperación y una persona responsable. Ajusta la profundidad según el riesgo: una página informativa y un sistema con pagos o datos sensibles no tienen las mismas necesidades.

Índice de lectura

Páginas	Contenido
3–5	Cómo trabajar con IA y validar un problema
6–9	Herramientas, entorno, contexto y requisitos
10–13	Arquitectura, diseño, reglas e implementación
14–17	Prompts, pruebas, revisión y seguridad
18–21	Automatización, datos, despliegue y operación
22–24	Diagnóstico, criterios de salida y fuentes

02 / FUNDAMENTOS

Entiende a tu colaborador de IA

Aquí usamos vibe coding como una puerta de entrada al desarrollo mediante instrucciones en lenguaje natural. Cuando el producto afecta a usuarios reales, añadimos prácticas de ingeniería: requisitos, control de cambios, verificación y operación.

Qué puede hacer un agente

Según la herramienta y los permisos, puede leer un repositorio, proponer un plan, editar archivos, ejecutar comandos y comprobar resultados. Su capacidad depende del modelo, del contexto disponible, de las herramientas y de los límites del entorno.

Qué no debes dar por hecho

Fallo posible	Respuesta práctica
API o paquete inexistente	Comprobar existencia y versión en la fuente oficial
Solución basada en un supuesto falso	Separar hechos, hipótesis y dudas antes de implementar
Cambios fuera de alcance	Definir límites y revisar el diff
Pruebas adaptadas al error	Revisar cambios en tests contra los requisitos
Verificación solo declarada	Pedir comandos, resultados y comprobaciones omitidas
Instrucciones maliciosas en archivos	Tratar contenido externo como datos y limitar herramientas

No confundas confianza verbal con certeza

Que el agente diga “resuelto” describe su conclusión, no una prueba. Solicita evidencia proporcionada al cambio y examina las consecuencias. Otra IA puede ayudar a revisar, pero también compartir el mismo supuesto equivocado.

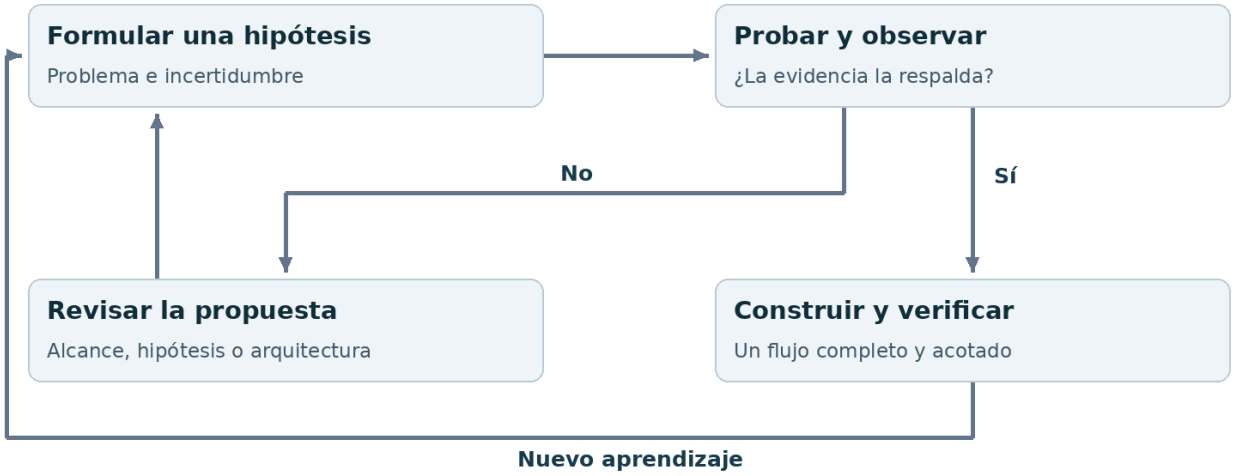
No existe un prompt que garantice el resultado

Un buen prompt reduce ambigüedad. Los permisos restringen acciones. Las pruebas comprueban comportamientos. La revisión evalúa decisiones y riesgos. Estos mecanismos se complementan; ninguno vuelve infalible al sistema. [1, 2, 3]

02 / FUNDAMENTOS

Aprende mediante ciclos cortos

El mapa es una orientación, no una cadena rígida. Una observación de usuario puede cambiar el alcance; una prueba técnica puede invalidar la arquitectura. Reserva espacio para volver atrás antes de invertir más trabajo.



Tres bucles que conviene distinguir

Bucle	Pregunta central	Qué revisas si falla
Producto	¿Esto resulta útil?	Problema, usuario, propuesta y alcance
Implementación	¿Hace lo acordado?	Código, datos, interacción y pruebas
Operación	¿Podemos sostenerlo?	Capacidad, costos, alertas y recuperación

Decisiones con evidencia

Para cada etapa registra: qué necesitas decidir, qué evidencia respalda la decisión y qué condición impide avanzar. La cantidad de documentos puede variar; la necesidad de comprobar lo importante permanece.

Decisión para avanzar

El siguiente paso reduce una incertidumbre concreta.

Evidencia necesaria. Una pregunta, un experimento acotado y una observación que puedas interpretar.

Detente si: Estás construyendo más funciones para evitar comprobar si la primera sirve.

03 / DESCUBRIR

Define y valida el problema

Cinco preguntas iniciales

- ¿Qué dificultad concreta quieres resolver?
- ¿Quién la tiene y en qué situación aparece?
- ¿Cómo se resuelve hoy?
- ¿Qué cambio útil debería producir tu solución?
- ¿Qué queda fuera de esta primera versión?

Ejemplo de trabajo

StudentHub explora cómo ayudar a estudiantes a encontrar sus pendientes académicos. La hipótesis es que reunir entregas próximas y su estado reduce la confusión. No damos esa utilidad por demostrada porque podamos guardar tareas.

MVP como prueba de una hipótesis

Primera versión posible: crear una tarea con fecha, ver pendientes próximos y marcar una tarea como completada. Los apuntes, la comunidad, los pagos y el tutor con IA pueden esperar. Si solo pruebas interacción con datos ficticios, quizá todavía no necesites cuentas.

Tipo de validación	Qué demuestra	Ejemplo
Funcional	El sistema hace una operación	La tarea se guarda y persiste
De utilidad	La persona logra su objetivo	Encuentra qué entregar a continuación
De producto	El uso aporta valor sostenido	Vuelve a utilizarlo por su utilidad

Investiga usuarios y viabilidad

Observa alternativas, dificultades, lenguaje y contexto de uso. Revisa integraciones, permisos, costos y restricciones técnicas. Usa la IA para organizar evidencia y preguntas; no presentes usuarios simulados o respuestas generadas como investigación real.

Decisión para avanzar

Vale la pena probar una solución acotada.

Evidencia necesaria. Observaciones reales, una hipótesis explícita y un flujo que permite evaluarla.

Detente si: Solo tienes una lista de funciones o desconoces una integración esencial.

04 / PREPARAR

Elige tecnologías por necesidad

La herramienta adecuada es la que permite resolver el trabajo con un nivel de control y mantenimiento que puedas asumir. Compara capacidades actuales en fuentes oficiales, no solo demos o promesas comerciales.

Necesidad	Categoría que conviene evaluar
Explorar una interfaz rápidamente	Constructores visuales o entornos en navegador
Trabajar en un repositorio existente	Editor o agente que inspeccione y modifique el código
Ejecutar tareas técnicas locales	Agente de terminal con permisos delimitados
Mantener un servicio	Entorno con código exportable, pruebas y despliegues rastreables

Replit, Lovable, Bolt, Cursor y Claude Code son ejemplos de herramientas para evaluar. Sus funciones, restricciones y precios cambian. Comprueba exportación, acceso al código, tratamiento de datos y control sobre el despliegue antes de depender de ellas.

Un stack posible para el ejemplo

Next.js con TypeScript, Tailwind CSS, PostgreSQL mediante Supabase, Supabase Auth cuando se necesiten cuentas, Git y GitHub, Playwright para recorridos de navegador y un hosting compatible como Vercel. Este conjunto es una opción, no una receta obligatoria.

Antes de agregar una tecnología

- Identifica la necesidad concreta que cubre.
- Comprueba documentación, compatibilidad y mantenimiento.
- Evalúa costos, exportación y dependencia del proveedor.
- Prefiere capacidades existentes cuando sean suficientes.

Decisión para avanzar

El stack es adecuado y comprensible para el equipo.

Evidencia necesaria. Una razón por elección y una prueba de las integraciones inciertas.

Detente si: La elección depende de una API no verificada o agrega complejidad sin utilidad clara.

04 / PREPARAR

Prepara un entorno reproducible

Para un proyecto web local necesitas un editor o agente, Git, un runtime compatible, un gestor de paquetes y un navegador. Docker, clientes de API y otras utilidades se agregan cuando el proyecto las requiera.

Comprueba las herramientas

```
node --version
npm --version
git --version
```

Registra la versión de Node usada y el gestor elegido. Conserva el archivo de bloqueo de dependencias. Evita mezclar gestores sin una decisión explícita.

Inicia el repositorio

```
mkdir student-dashboard
cd student-dashboard
git init
```

Estos comandos no generan la aplicación. Sigue el procedimiento oficial de inicialización del framework elegido y revisa las opciones antes de ejecutar comandos sugeridos. Después configura el repositorio remoto.

Comprueba que se pueda reconstruir

Desde un checkout limpio, otra persona o la CI debe poder instalar las dependencias, configurar un entorno de prueba y ejecutar las comprobaciones documentadas. Para npm, npm ci requiere un lockfile compatible y puede usarse en ese recorrido.

```
npm ci
npm run lint
npm run typecheck
npm test
npm run build
```

Los scripts son ejemplos: deben existir en package.json. No inventes un resultado satisfactorio cuando una comprobación no está configurada. Registra qué falta y decide si impide avanzar.

Decisión para avanzar

El proyecto se instala y verifica fuera de tu sesión habitual.

Evidencia necesaria. Versiones registradas, lockfile y ejecución limpia de los comandos disponibles.

Detente si: Solo funciona gracias a archivos, credenciales o pasos locales sin documentar.

04 / PREPARAR

Organiza el contexto con criterio

El objetivo es aportar información relevante y vigente, no maximizar archivos o tokens. Empieza con una entrada breve y carga los detalles cuando la tarea los necesite. [1]

Documento	Contenido	Cuándo resulta útil
README.md	Instalación, comandos y mapa del proyecto	Siempre
PRD.md	Problema, hipótesis, alcance y aceptación	Al definir el producto
Instrucciones del agente	Reglas, límites y referencias	Al trabajar con IA
TASKS.md	Tareas y criterios de finalización	Durante la implementación
ARCHITECTURE.md	Límites y responsabilidades	Cuando hay decisiones técnicas relevantes
DESIGN.md	Interacción y sistema visual	Cuando se necesita consistencia
DECISIONS.md	Decisión, motivo y consecuencias	Para elecciones importantes
TEST_PLAN.md y SECURITY.md	Comprobaciones y riesgos	Al introducir comportamientos o datos críticos
MEMORY.md	Estado breve y pendientes	Para retomar sesiones largas

Carga selectiva

Una tarea visual necesita referencias de interfaz y componentes. Un cambio de permisos necesita el modelo de acceso y sus pruebas. El agente puede consultar más archivos si identifica una dependencia; no debe asumir que lo no leído carece de importancia.

Memoria verificable y fuentes de verdad

Incluye fecha o revisión de referencia, tarea actual, resultados comprobados y asuntos pendientes. Contrasta el resumen con código, pruebas y decisiones vigentes. Si se contradicen, resuelve la diferencia antes de programar. La memoria no es una garantía de lo sucedido.

Decisión para avanzar

El agente puede localizar el contexto necesario.

Evidencia necesaria. Una entrada breve, referencias claras y estado contrastado.

Detente si: Hay instrucciones contradictorias o documentos que describen un producto inexistente.

04 / PREPARAR

Escribe requisitos verificables

El PRD explica qué quieres conseguir y por qué. Mantén separados el problema, la hipótesis de utilidad y los comportamientos que la implementación debe cumplir.

```
# PRD — StudentHub

## Problema observado
Los estudiantes consultados tienen dificultades para identificar
qué entrega deben atender a continuación.

## Hipótesis
Una lista ordenada de pendientes puede reducir esa dificultad.

## Usuario y contexto
Estudiantes que gestionan entregas de varias asignaturas.

## Primera versión
- Crear tarea con título y fecha
- Ver pendientes próximos
- Marcar tarea como completada

## Fuera de alcance
Pagos, tutor con IA, comunidad y aplicación móvil nativa.

## Aceptación funcional
La tarea válida se guarda y permanece después de recargar.
Las entradas inválidas muestran un mensaje comprensible.
Si hay cuentas, cada persona accede solo a sus tareas.

## Validación de utilidad
Observar si las personas identifican su siguiente entrega
sin ayuda y qué obstáculos encuentran.

## Incertidumbres
Confirmar cómo quieren ordenar entregas con la misma fecha.
```

Este es un ejemplo de estructura: sustituye las afirmaciones sobre usuarios por evidencia real. Define antes de la prueba qué observarás y cómo interpretarás los resultados. No inventes porcentajes de éxito ni resultados de investigación.

Decisión para avanzar

El equipo entiende qué construir y cómo evaluarlo.

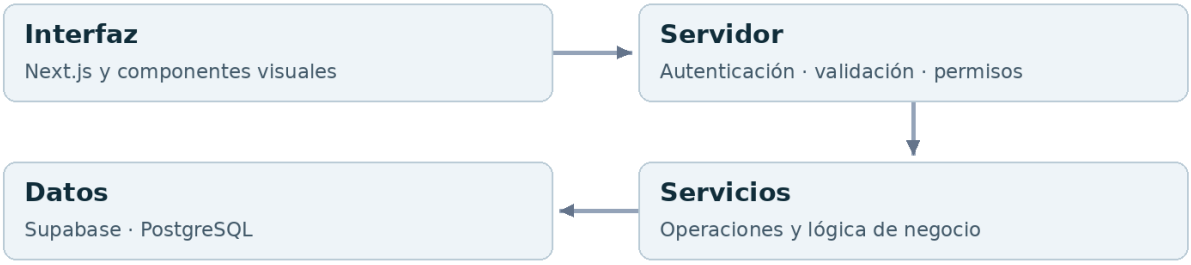
Evidencia necesaria. Alcance explícito, criterios comprobables y plan de validación.

Detente si: La única medida de éxito es terminar pantallas o generar código.

05 / DISEÑAR

Diseña responsabilidades y datos

La arquitectura explica los límites del sistema. Antes de multiplicar carpetas, identifica qué código llega al navegador, qué operaciones requieren servidor y dónde se aplican permisos.



El diagrama muestra una opción con servidor y capa de servicios. Otras arquitecturas pueden permitir acceso del cliente a datos mediante políticas de acceso. Elige una y documenta sus límites de confianza; no mezcles modelos por accidente.

Modelo mínimo del ejemplo

Entidad o regla	Definición posible
Tarea	id, título, fecha, estado y propietario si hay cuentas
Propiedad	El propietario se obtiene de la identidad validada
Acceso	Lectura y modificación limitadas al usuario autorizado
Tiempo	Definir fecha, zona horaria y criterio de orden
Integridad	Campos requeridos y valores válidos

Estructura proporcional

Usa app/, components/, features/ y otras carpetas si clarifican responsabilidades. No crees services/, lib/, types/ y utils/ vacíos por cumplir una plantilla. Evita duplicar lógica o dispersar una función sin una razón práctica.

Decisión para avanzar

Puedes explicar el recorrido y la protección de los datos.

Evidencia necesaria. Modelo, límites y una decisión técnica breve.

Detente si: El navegador puede enviar un propietario arbitrario o los permisos dependen de ocultar botones.

05 / DISEÑAR

Diseña uso y estados completos

Un sistema visual útil define también cómo interactúa la persona, qué información necesita y cómo se recupera de errores. Comienza por el flujo principal antes de ampliar la biblioteca de componentes.

Aspecto	Qué definir
Jerarquía	Qué es principal, secundario y contextual
Tipografía y color	Legibilidad, escalas y contraste en su uso real
Componentes	Variantes, estados y comportamiento
Formulario	Etiquetas, validación y mensajes útiles
Responsive	Reordenamiento y contenido según espacio
Accesibilidad	Teclado, foco visible y nombres comprensibles

Todos los estados forman parte del producto

- Carga: mostrar progreso sin permitir acciones duplicadas.
- Vacío: explicar cómo empezar.
- Error: comunicar qué ocurrió y cómo continuar.
- Éxito: confirmar el resultado sin depender solo del color.
- Sesión caducada: recuperar el acceso sin perder información innecesariamente.

Ejemplo de DESIGN.md

```
Estilo: claro, sobrio y orientado a tareas.
Tipografía: una familia legible con escala consistente.
Componentes: reutilizar los existentes.
Acción principal: crear una tarea.
Acción destructiva: confirmar cuando corresponda al riesgo.
Estados: carga, vacío, error y éxito.
Revisión: teclado, móvil y contenido real.
```

Decisión para avanzar

El flujo se entiende y se puede recorrer.

Evidencia necesaria. Prueba con contenido representativo, teclado y pantallas relevantes.

Detente si: La interfaz solo funciona con datos ideales o no explica cómo recuperarse de un error.

05 / DISEÑAR

Limita acciones y conserva decisiones

Tres mecanismos distintos

Mecanismo	Ejemplo	Qué aporta
Instrucción	No cambiar el esquema sin autorización	Orienta al agente
Restricción	Credenciales limitadas y aislamiento	Acota lo que puede hacer
Evidencia	Diff y comprobación de permisos	Permite evaluar lo realizado

No sustituyas una restricción técnica por una frase en RULES.md. Para un agente que ejecuta comandos, evalúa permisos de archivos, red e integraciones. Usa acceso mínimo y evita credenciales de producción en el entorno de desarrollo. [2]

Reglas de trabajo breves

- Inspecciona lo existente y conserva los cambios ajenos.
- No cambies el alcance ni agregues dependencias sin justificarlo.
- No ejecutes operaciones destructivas ni publiques por inferencia.
- Comprueba las APIs y versiones que no conozcas.
- No ocultes fallos desactivando pruebas o controles.
- Informa resultados, limitaciones y asuntos pendientes.

Usa el archivo de instrucciones que tu herramienta reconozca y enlaza documentación adicional. RULES.md por sí solo no se carga en todos los editores. Confirma el comportamiento en la documentación de la herramienta.

Registra decisiones importantes

```
ADR-001
Decisión: usar un servicio de autenticación mantenido.
Motivo: el equipo no desarrollará su propio sistema de claves.
Alternativas consideradas: ...
Consecuencias: dependencia del proveedor y configuración de acceso.
Revisar si: cambian los requisitos de identidad o exportación.
```

Decisión para avanzar

Las acciones permitidas están delimitadas.

Evidencia necesaria. Reglas comprensibles y permisos técnicos acordes al trabajo.

Detente si: El agente tiene acceso innecesario a datos o puede publicar cambios críticos sin revisión.

06 / IMPLEMENTAR

Construye una unidad verificable

El tamaño correcto de una tarea depende del comportamiento y el riesgo. Una función completa puede ser más coherente que dividir artificialmente cada campo, archivo y test en conversaciones separadas.

Corte vertical del ejemplo

Crear tarea → validar → guardar con permisos → recuperar → mostrar → recargar y comprobar persistencia. Este recorrido conecta interfaz, lógica y datos para descubrir fallos de integración pronto.

Plantilla de tarea

```
TASK-012 — Crear una tarea

Objetivo: registrar una entrega con título y fecha.
Alcance: formulario, operación de guardado y lista actualizada.
Fuera de alcance: recordatorios, pagos y tareas compartidas.
Dependencias: modelo y permisos definidos.

Aceptación:
- Datos válidos se guardan y persisten al recargar.
- Datos inválidos no generan registros.
- La persona recibe un mensaje útil si falla el guardado.
- Si hay cuentas, no se puede crear para otro propietario.

Evidencia:
- Pruebas del comportamiento
- Recorrido manual o E2E
- Diff revisado y limitaciones informadas
```

Cuándo dividir más

Divide si la tarea acumula decisiones independientes, afecta varias áreas críticas o no se puede revisar con claridad. Durante el aprendizaje puede convenir implementar primero una interfaz con datos ficticios; después completa la integración sin perder el objetivo del flujo.

Decisión para avanzar

La unidad está terminada y revisada.

Evidencia necesaria. Criterios satisfechos y comprobaciones registradas.

Detente si: La tarea solo parece completa porque se ocultaron errores o se dejó pendiente la protección de datos.

06 / IMPLEMENTAR

Da contexto y pide evidencia

Prompt antes de modificar

```
Lee README.md y las instrucciones del agente.
Consulta los requisitos y documentos pertinentes a TASK-012.
Inspecciona la implementación existente.

Todavía no modifiques archivos.
Resume el comportamiento esperado, los límites del cambio
y las dudas que afecten a la solución.
Propón un plan proporcional al riesgo y cómo comprobarlo.
```

Prompt de implementación

```
CONTEXTO
StudentHub necesita registrar tareas con fecha.

TAREA
Implementa TASK-012 según sus criterios de aceptación.

ARCHIVOS
Localiza los módulos relevantes; conserva el trabajo existente.

RESTRICCIONES
Reutiliza componentes y la capa de datos acordada.
No amplíes el alcance ni debilites pruebas o permisos.
No hagas push ni despliegues salvo autorización explícita.

VERIFICACIÓN
Ejecuta las comprobaciones pertinentes ya configuradas.
Si falta alguna, explica su efecto sobre la validación.

ENTREGA
Informa qué cambió y por qué, archivos modificados, comandos,
resultados observados, limitaciones y asuntos sin comprobar.
```

Usa las pausas donde ayuden

Un cambio trivial y reversible puede ejecutarse directamente dentro del alcance autorizado. Un cambio de datos, permisos o arquitectura merece revisar el plan. Evita convertir cada acción pequeña en una aprobación sin contenido.

07 / VERIFICAR

Prueba comportamientos y límites

Las pruebas deben derivarse del comportamiento esperado. Si el agente escribe implementación y tests, revisa que ambos no estén confirmando el mismo supuesto equivocado.

Nivel	Uso útil	Ejemplo
Estático	Detectar errores sin ejecutar el flujo	Tipos y lint
Unitario	Comprobar una regla aislada	Orden de fechas
Integración	Comprobar relaciones entre partes	Guardar con permisos
E2E	Recorrer una acción desde el navegador	Crear y recuperar una tarea
Manual con usuarios	Observar comprensión y utilidad	Identificar la siguiente entrega

Incluye casos adversos

- Solicitud sin sesión o con sesión vencida.
- Intento de leer o modificar el recurso de otra persona.
- Campos vacíos, valores inválidos y entradas inesperadas.
- Fallo de red o del servicio de datos.
- Doble envío y recarga de la página.

No pruebes todo al mismo nivel

Prioriza recorridos y reglas de mayor riesgo. No agregues tests que solo repitan la estructura del código. Tampoco uses cobertura numérica como sustituto de comprobar permisos y comportamientos importantes.

Evidencia de ejecución

Registra comandos, entorno, resumen de resultados y fallos pendientes. Un test no ejecutado debe figurar como no ejecutado. Los logs pueden requerir ocultar datos sensibles antes de compartirlos.

Decisión para avanzar

La evidencia respalda el comportamiento acordado.

Evidencia necesaria. Pruebas relevantes y casos negativos revisados.

Detente si: Se cambiaron expectativas para aceptar el fallo o faltan comprobaciones críticas.

07 / VERIFICAR

Revisa cambios y usa Git

La revisión con IA es una ayuda adicional. La persona responsable conserva el juicio sobre requisitos, riesgos y publicación. Una segunda IA tampoco garantiza independencia de criterio. [3]

Qué revisar en el diff

- Relación entre cada cambio y la tarea acordada.
- Permisos, manejo de errores y consecuencias sobre datos.
- Pruebas nuevas y modificaciones en expectativas existentes.
- Dependencias agregadas y código duplicado.
- Cambios ajenos, secretos o desactivación de controles.

Conserva versiones comprensibles

```
git status
git diff
git add src/features/tasks/ tests/
git diff --staged
git commit -m "feat: add task creation"
```

Las rutas son ejemplos: selecciona las que correspondan. Revisa el contenido preparado y respeta la autorización de commits y push. Usa ramas para aislar cambios importantes y un proceso de revisión antes de integrarlos.

Qué recupera Git

Git permite recuperar versiones de archivos registrados. No restaura por sí mismo registros borrados de una base de datos, archivos externos ni configuración de servicios. El plan de recuperación debe contemplar cada recurso afectado.

Decisión para avanzar

El cambio está listo para integrarse.

Evidencia necesaria. Diff comprendido, controles relevantes satisfactorios y decisiones pendientes resueltas.

Detente si: La única revisión es la afirmación del mismo agente que implementó el cambio.

07 / VERIFICAR

Protege a usuarios y al agente

Seguridad de la aplicación

Autenticación responde quién es la persona; autorización, qué puede hacer. Verifica las dos en cada operación protegida. Una pantalla oculta o una ruta visualmente bloqueada no sustituyen permisos sobre los datos.

Riesgo	Control y comprobación
Acceso a datos ajenos	Permisos en la operación y pruebas con dos usuarios
Entradas inesperadas	Validación y manejo seguro en límites de confianza
Abuso del servicio	Límites proporcionales, alertas y control de consumo
Carga de archivos	Validación de tipo, tamaño y acceso cuando exista
Exposición de información	Datos mínimos y logs sin credenciales ni contenido sensible

Seguridad del entorno de IA

Revisa herramientas, plugins y conexiones antes de conceder acceso. Limita archivos, red y credenciales a lo que necesita la tarea. Contenido de un README, una página o una salida de herramienta puede contener instrucciones maliciosas; no lo conviertas en autorización. El aislamiento reduce el alcance de un error o ataque, sin eliminar todo riesgo. [2]

Dependencias y ejecución

Comprueba nombre, procedencia y mantenimiento de paquetes antes de instalarlos. Revisa alertas de vulnerabilidades y evita ejecutar scripts desconocidos por una sugerencia sin verificar. Considera qué código puede ejecutarse durante instalación, pruebas y despliegue.

Decisión para avanzar

Los límites de acceso están comprobados.

Evidencia necesaria. Pruebas de autorización y permisos mínimos del agente y los servicios.

Detente si: Hay una ruta de acceso sin verificar o credenciales privilegiadas innecesarias.

Automatiza comprobaciones repetibles

La integración continua o CI ejecuta controles en un entorno definido. Ayuda a detectar fallos sin depender del recuerdo de ejecutar comandos en tu computadora. GitHub Actions permite automatizar comprobaciones sobre pull requests y otros eventos. [5]

Pipeline mínimo para el ejemplo

Paso	Qué comprueba
Instalar desde el lockfile	Dependencias del estado versionado
Lint y tipos	Errores estáticos según la configuración
Pruebas pertinentes	Reglas e integración del sistema
Build	Compilación de la versión que se publicará
Preview y pruebas de humo	Recorrido esencial en el entorno desplegado

Haz que los controles importen

Configura las comprobaciones requeridas en el proceso de integración. Un workflow que falla pero se ignora no bloquea una publicación defectuosa. No agregues secretos de producción a trabajos que no los necesiten ni a contextos de código no confiable.

Entorno y credenciales

```
# Ejemplo de configuración pública para Next.js y Supabase
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY=

# Solo si el servidor necesita operaciones privilegiadas
SUPABASE_SECRET_KEY=
```

Guarda una plantilla sin valores reales en .env.example. Excluye los archivos locales con secretos mediante .gitignore y configura los valores por entorno en el hosting.

La clave publishable está destinada al cliente y necesita políticas de acceso adecuadas. La clave secret es privilegiada y no debe enviarse al navegador ni registrarse en Git. No es necesaria en todo proyecto. Los nombres y prefijos dependen del framework. [4]

Decisión para avanzar

El cambio puede verificarse automáticamente.

Evidencia necesaria. Ejecución rastreable de la CI sobre la revisión propuesta.

Detente si: Los controles críticos se omiten o las credenciales no corresponden al entorno.

Versiona datos y prepara recuperación

Una migración registra un cambio del esquema de datos. Conserva esas instrucciones en el repositorio y prueba su aplicación sobre un entorno de ensayo. Evita cambios manuales que nadie pueda reconstruir. [6]

Antes de modificar la base de datos

- Identifica el impacto sobre registros existentes.
- Comprueba duración, bloqueos y compatibilidad de versiones.
- Ensaya la migración con datos representativos y protegidos.
- Define qué harás si falla a mitad del proceso.
- Verifica una copia de seguridad apropiada al cambio.

Separa recuperación de código y datos

Recurso	Mecanismo posible	Evidencia
Código	Volver a una versión validada	Versión identificada y recorrido comprobado
Base de datos	Restaurar backup o corregir mediante migración	Ensayo de recuperación y comprobación de integridad
Archivos externos	Backup y restauración del almacenamiento	Archivos recuperados con permisos correctos
Configuración	Inventario protegido por entorno	Valores necesarios identificados sin exponer secretos

Restaurar datos puede perder cambios posteriores al backup. Define cuánta pérdida es tolerable y cuánto tiempo puede estar interrumpido el servicio. No reviertas credenciales comprometidas a un valor viejo: rótalas y revoca lo expuesto.

Decisión para avanzar

Existe una salida practicable si el cambio falla.

Evidencia necesaria. Migración ensayada, copia recuperable y responsable de recuperación.

Detente si: Solo sabes volver al commit anterior o nunca has comprobado la restauración.

09 / DESPLEGAR

Publica una versión identificable

Local → preview → QA → producción es una ruta común. En cada paso conserva la relación entre código, configuración y cambios de datos. Una URL accesible no demuestra que el despliegue esté listo.

Aísla los entornos

Entorno	Datos y uso
Desarrollo	Datos ficticios o de prueba
Preview	Ensayo sin acceso accidental a datos reales
Producción	Usuarios reales y permisos operativos controlados

Verifica la preview

Prueba el recorrido principal, errores, permisos, enlaces directos, recarga, móvil y teclado. Comprueba variables, redirecciones de autenticación y conexiones a datos. Verifica que cada dependencia apunta al entorno correcto.

Registro breve de publicación

```
Versión o commit: ...
Entorno y configuración verificados: ...
Migraciones necesarias: ...
Comprobaciones y resultados: ...
Problemas conocidos aceptados: ...
Responsable de publicación y recuperación: ...
Plan si falla: ...
```

Después de publicar

Ejecuta pruebas de humo sobre la URL real sin dañar datos de usuarios. Observa errores y rendimiento. Si la versión falla, evalúa si basta recuperar código o si también hay que corregir datos o configuración.

Decisión para avanzar

La versión puede recibir el uso previsto.

Evidencia necesaria. Preview validada, versión rastreable, responsable y plan de recuperación.

Detente si: Hay controles críticos fallidos o no sabes qué revisión y qué datos estás publicando.

10 / MANTENER

Opera y aprende del uso real

Operar significa detectar problemas, responder a ellos y sostener el servicio. No se resuelve instalando un panel de analítica: define qué señales importan y quién actuará.

Señal	Pregunta que ayuda a responder
Errores	¿Qué flujo está fallando y desde cuándo?
Disponibilidad	¿La aplicación responde?
Rendimiento	¿La experiencia se degrada?
Datos y backups	¿Podemos recuperar información?
Consumo y costos	¿Estamos dentro del presupuesto?
Feedback y uso	¿El producto sigue siendo útil?

Presupuesto y alertas

Identifica costos fijos y variables: hosting, base de datos, almacenamiento, correo y APIs. Configura alertas y límites cuando el proveedor los permita. Una alerta de gasto no siempre detiene el consumo: comprueba su comportamiento.

Respuesta básica a incidentes

Detectar → dimensionar impacto → contener → recuperar → verificar → comunicar lo necesario → registrar aprendizaje. Si hay exposición de credenciales, revócalas o rótalas; borrar el archivo no neutraliza una clave filtrada.

Mantenimiento y documentación

Revisa dependencias, alertas y políticas de acceso con una frecuencia acorde al riesgo. Actualiza README, decisiones, tareas y contexto cuando cambie el sistema. Revisa también si el producto merece seguir creciendo, mantenerse o retirarse.

Decisión para avanzar

El servicio tiene una operación sostenible.

Evidencia necesaria. Responsable, alertas accionables, recuperación ensayada y presupuesto conocido.

Detente si: Nadie recibe alertas o una caída depende de que un usuario la denuncie por casualidad.

11 / RESOLVER Y RETOMAR

Depura sin ampliar el problema

Describe el fallo de forma reproducible

```
ERROR
Mensaje y contexto, sin secretos.

ESPERADO
La tarea debería guardarse y aparecer en la lista.

OBSERVADO
La interfaz confirma el guardado, pero al recargar desaparece.

REPRODUCCIÓN
1. Iniciar sesión con una cuenta de prueba.
2. Crear una tarea válida.
3. Recargar y revisar el resultado.

RESTRICCIONES
No cambies el esquema ni los permisos sin justificarlo.

PRIMER PASO
No modifiques código todavía. Identifica hipótesis, evidencia
y la comprobación más pequeña para distinguir la causa.
```

Corrige después del diagnóstico

Pide la corrección mínima coherente con la causa y una prueba que reproduzca el fallo. Si se necesita cambiar el alcance o la arquitectura, hazlo explícito. Evita encadenar arreglos especulativos o desactivar controles para obtener un resultado en verde.

Retoma una sesión con contexto breve

```
Estado comprobado: ...
Revisión de código o versión: ...
Tarea actual y alcance: ...
Últimas pruebas y resultados: ...
Hipótesis pendientes: ...
Siguiente comprobación: ...
```

El código, las decisiones y la ejecución real deben respaldar este resumen. Cuando haya contradicciones, investiga antes de confiar en la memoria del agente.

12 / REFERENCIA PRÁCTICA

Comprueba antes de avanzar

Recorrido	Evidencia mínima para pasar a otro nivel
Exploración	Problema e hipótesis explícitos; datos ficticios; observación de utilidad; límites del prototipo visibles
MVP con usuarios	Flujo útil; aceptación funcional; permisos probados si hay cuentas; configuración separada; responsable y vía de feedback
Producto en operación	Controles automáticos; versión rastreable; migraciones ensayadas; recuperación comprobada; alertas, presupuesto y responsable

Por cada función

- Entender el comportamiento y las restricciones.
- Consultar el contexto pertinente y lo ya implementado.
- Planificar según la incertidumbre y el riesgo.
- Implementar una unidad revisable.
- Probar comportamientos y casos negativos relevantes.
- Revisar el diff y la evidencia de ejecución.
- Registrar una versión según la política del proyecto.
- Actualizar el contexto que realmente cambió.

Señales para detenerse

Permisos sin comprobar, secretos expuestos, datos de producción en una prueba accidental, controles críticos desactivados, migraciones no ensayadas o afirmaciones de calidad sin evidencia. Resuelve la causa antes de continuar.

Glosario breve

MVP: primera versión para validar utilidad. Stack: tecnologías del proyecto. PRD: requisitos del producto. ADR: registro de una decisión técnica. Diff: diferencias entre versiones. CI: integración continua. E2E: recorrido de extremo a extremo. Preview: despliegue previo. QA: verificación de calidad. RLS: políticas de acceso por fila. Rollback: recuperación de una versión anterior, cuyo alcance depende del recurso.

El criterio final no es cuánto código produjo la IA. Es qué problema resolviste, qué comprobaste y qué puedes sostener con responsabilidad.

Fuentes y alcance de la guía

Esta edición reorganiza y amplía el material inicial con criterios editoriales de desarrollo de producto e ingeniería. Las plantillas son ejemplos adaptables. Las fuentes siguientes respaldan aspectos concretos sobre agentes, contexto, acceso a datos y automatización; no certifican la seguridad de un proyecto particular.

1 Ingeniería del contexto

Anthropic · Effective context engineering for AI agents. Contexto pertinente, notas y continuidad de tareas.
https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents

2 Límites del entorno del agente

Anthropic · Beyond permission prompts making Claude Code more secure and autonomous. Aislamiento de archivos y red y riesgos de inyección de instrucciones.
https://www.anthropic.com/engineering/claude-code-sandboxing

3 Revisión humana y límites de agentes

GitHub Docs · Application card GitHub Copilot Agents. Verificación de resultados y revisión con IA como complemento.
https://docs.github.com/en/copilot/responsible-use/agents

4 Claves y acceso a Supabase

Supabase Docs · API keys. Diferencia entre claves publicables, secretas y acceso mediante políticas.
https://supabase.com/docs/guides/getting-started/api-keys

5 Integración continua

GitHub Docs · Understanding GitHub Actions. Automatización de compilación y pruebas.
https://docs.github.com/en/actions/get-started/understand-github-actions

6 Migraciones de datos

Supabase Docs · Deploy database migrations. Registro y aplicación de cambios de esquema.
https://supabase.com/docs/guides/deployment/database-migrations

Fuentes consultadas el 30 de octubre de 2026. Verifica versiones y condiciones de los servicios antes de aplicar comandos o configuraciones. Para productos con riesgos altos o requisitos sectoriales, agrega revisión especializada proporcional a su contexto.